12 · Template-Driven Forms

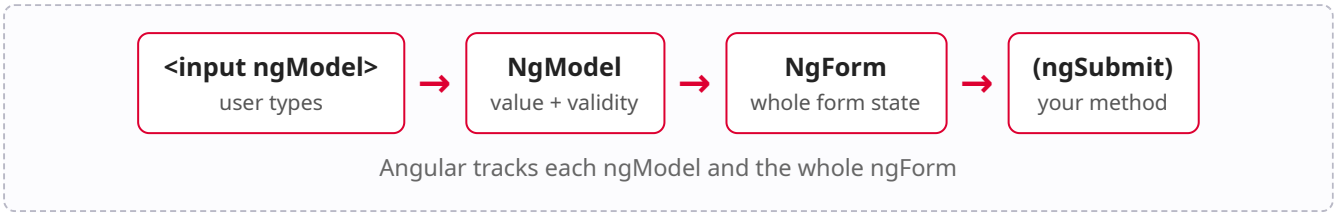
Intermediate Angular learning notes

a. Definition

In a **template-driven form** you describe the form mostly in HTML. Add `FormsModule`, put `ngModel` and a `name` on each input, and Angular builds the form model behind the scenes.

Add validation with HTML attributes like `required`, `minlength` and `email`. Template reference variables such as `#f="ngForm"` and `#nameCtrl="ngModel"` give you the state: `valid`, `invalid`, `dirty`, `touched` and `errors`. This style is good for small, simple forms.

b. Diagram



Angular tracks each ngModel and the whole ngForm

c. Syntax

```
<form #f="ngForm" (ngSubmit)="save(f.value)">
  <input name="email" [(ngModel)]="model.email" required email #e="ngModel" />
  @if (e.invalid && e.touched) { <small>Invalid email</small> }
  <button [disabled]="f.invalid">Save</button>
</form>
```

d. Example Program

Programs/12_Template_Driven_Forms/register-form.component.ts

```
import { Component } from '@angular/core';
import { FormsModule, NgForm } from '@angular/forms';

@Component({
  selector: 'app-root',
  imports: [FormsModule],
  template: `
    <h3>Course Registration</h3>
    <form #f="ngForm" (ngSubmit)="submit(f)">
      <label>Name
        <input id="name" name="name" [(ngModel)]="model.name" required minlength="3"
#nameCtrl="ngModel" />
      </label>
      @if (nameCtrl.invalid && (nameCtrl.dirty || nameCtrl.touched)) {
        <small>
          @if (nameCtrl.errors?.['required']) { Name is required. }
          @if (nameCtrl.errors?.['minlength']) { Use at least 3 letters. }
        </small>
      }

      <label>Email
        <input id="email" name="email" type="email" [(ngModel)]="model.email" required email
/>
```

```
      </label>

      <button type="submit" [disabled]="f.invalid">Register</button>
    </form>

    <p>Form valid: {{ f.valid }}</p>
    @if (message) { <p>{{ message }}</p> }
  `,
})
export class RegisterFormComponent {
  model = { name: '', email: '' };
  message = '';

  submit(form: NgForm) {
    this.message = `Registered ${form.value.name} (${form.value.email})`;
    form.resetForm();
  }
}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Type `Ra` in the name box (too short).
2. Type a valid name and email, then click **Register**.

```
--- Name too short ---
Course Registration
Name  Use at least 3 letters. Email Register
Form valid: false

--- After submit ---
Course Registration
Name Email Register
Form valid: false
Registered Ravi (ravi@example.com)
```

f. Real-World Example (English + தமிழ்)

English

A railway reservation form is printed with boxes for name and age. If you hand it over with the name box empty, the counter rejects it. Required/minlength attributes act like that counter clerk.

தமிழ்

ரயில்வே முன்பதிவுப் படிவத்தை எடுத்துக்கொள்ளுங்கள். பெயர், வயது, பயண நாள் என்று ஒவ்வொரு கட்டமும் அச்சிடப்பட்ட காகிதம் அங்கேயே இருக்கிறது. பெயர் கட்டத்தை நிரப்பாமல் கவுண்டரில் கொடுத்தால், 'பெயர் எழுதுங்கள்' என்று உடனே திருப்பிக் கொடுத்துவிடுவார்கள்.

Template-driven form-இல் படிவத்தின் அமைப்பு HTML-இலேயே இருக்கும் (`required`, `minlength`). எல்லாக் கட்டங்களும் சரியாக இருந்தால் மட்டுமே **Register** பொத்தான் வேலை செய்யும் - கவுண்டர் ஊழியர் சரிபார்த்த பிறகே ஏற்றுக்கொள்வது போல.

g. Pros

- Fast to write for simple forms
- Little TypeScript code
- Familiar HTML validation attributes

h. Cons

- Hard to test
- Not suitable for dynamic or large forms
- Logic spread across the template

i. Advanced / New Features

- Signal Forms (experimental in v21) aim to simplify form handling
- Unified control state events (v18)

j. Common Mistakes

- Forgetting the `name` attribute on an `ngModel` input; the control will not register in the form.
- Not importing `FormsModule`.
- Showing errors immediately on a pristine form; check `dirty` or `touched` first.
- Using template-driven forms for big, dynamic forms where reactive forms are easier to test.

k. Interview Questions

Q1. What are template-driven forms?
Forms built mainly in the template with `ngModel`, where Angular creates the form model automatically.

Q2. Why is `name` needed on inputs inside a form?
NgForm registers each `ngModel` control by its `name` attribute.

Q3. What do `dirty`, `touched`, `valid` mean?
`dirty`: value changed; `touched`: input was focused then left; `valid`: all validators pass.

Q4. Template-driven vs reactive?
Template-driven is simpler and implicit; reactive is explicit, more scalable and easier to unit-test.

I. Practice Exercises

1. Build a feedback form with name, rating (select) and comments (required, minimum 10 characters).
2. Disable the submit button until a 'I agree' checkbox is ticked.
3. Show a red border on invalid inputs using `[class.invalid]`.